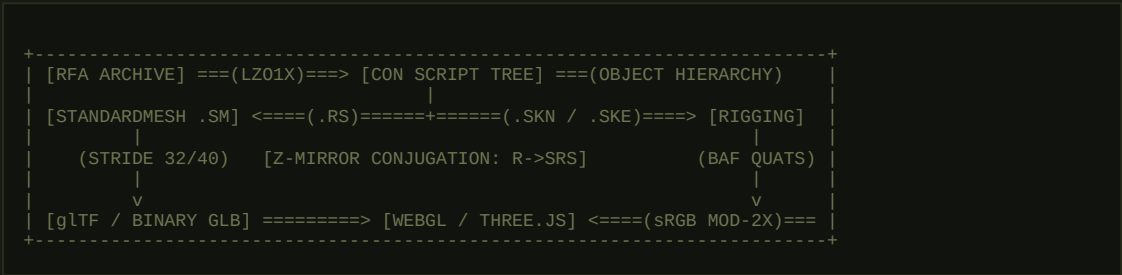


DISSECTING REFRACTOR

The Battlefield 1942 Asset Extraction & WebGL Reconstruction Field Guide

DIRECTX 8 • LZ01X • BIND POSES • THREE.JS SHADERS



A field manual for the technical developer. Explaining how raw binary meshes, nested template trees, run-length animation clips, and pre-lit terrains are extracted from 20-year-old game archives and rendered with mathematical fidelity in modern browsers.

PREFACE: NO SDK, JUST BYTES

Battlefield 1942 shipped in September 2002 on the Refractor 2 engine. There was no clean public SDK, no glTF export button, and no tidy JSON scene graphs. The engine is a creature of early DirectX 8 fixed-function architecture: flat uncompressed/LZO archives, stateful console scripts, custom vertex formats with proprietary strides, and MAX-exported bone hierarchies.

When you attempt to extract these models for the web, standard 3D tools fail. The mesh files contain no textures. The weapon skeletons are stored mirrored backwards along the Z-axis. The animation files transpose quaternions and yaw 180 degrees. The terrain is pre-lit with 2002 sun angles, so throwing standard Three.js hemisphere lighting onto it turns tropical atolls into dark grey mud.

This guide is written for your morning commute. It skips corporate fluff and cuts directly to the byte layouts, the math, the parsing bugs that burned hours of triage, and the WebGL tricks that brought 96 vanilla vehicles and thousands of mod assets into a browser at 60 FPS.

[★] THE REVERSE ENGINEERING GOLDEN RULE

Measure before you decompile. When a mesh looks broken, do not burn a day in Ghidra tracing stripped MSVC binaries. Write a Python survey across all 33,000 meshes in the installed mods. 95% of apparent engine quirks are either one-off authoring bugs in single mod files or simple coordinate-system misunderstandings.

THE RECONSTRUCTION PIPELINE AT A GLANCE

```
1. ARCHIVES (.rfa)      Unpack data payloads with LZ01X segment decompression.
2. OBJECT SCRIPTS (.con) Replay stateful command streams to build the parent-child scene graph.
3. GEOMETRY (.sm)      Read 16-bit indices, bounding boxes, collision hulls, and vertex strides.
4. SHADERS (.rs)       Match mesh material names to texture stages and alpha-test flags.
5. TEXTURES (.dds/.tga) Decode DXT1/3/5 and BGRA rasters; bleed cutouts to eliminate black halos.
6. SKELETONS (.ske/.skn) Conjugate Z-mirrored bone matrices (R -> SRS); weld sleeves to hands.
7. ANIMATION (.baf)    Decode 7-channel fixed-point RLE tracks; conjugate quaternions; yaw root 180°.
8. EXPORT (glTF / .glb) Transform Refractor Left-Hand to WebGL Right-Hand; bake node hierarchy.
9. BROWSER (Three.js)  Modulate-2x terrain shader; dual-UV lightmaps; interactive vehicle rigging.
```

THE 4-WAY HANDSHAKE OF A REFRACTOR ASSET

If you search for `Sherman.3ds` or `Tiger.obj` in Battlefield 1942, you will find nothing. A Refractor asset is not a file; it is a **four-way contract** spread across distinct archive domains:

```
+-----+ +-----+ +-----+
| Objects.con | --> | Geometries.con | --> | StandardMesh/*.sm |
| (Tree & Hierarchy) | | (Template Indirection) | | (Indexed Triangles) |
+-----+ +-----+ +-----+
                        |
                        v
+-----+ +-----+ +-----+
| Textures (*.dds) | <-- | RenderShader (*.rs) | <-----+
| (DXT1/3/5 Pixels) | | (Material Shaders) |
+-----+ +-----+ +-----+
```

- Objects.con** declares the hierarchy: *"A Sherman is a Bundle containing a Hull, parenting a Turret, parenting a Cannon."*
- Geometries.con** resolves geometry aliases: `ObjectTemplate.geometry Sherman_Hull_M1` maps to `StandardMesh/Sherman_Hull_M1.sm`.
- *.sm** stores raw vertex positions, normals, UVs, and material slot names (e.g. `Material0`). It knows nothing about textures.
- *.rs** (RenderShader) matches the material slot name to an image filename (e.g. `texture/sherma_I`).
- *.dds / *.tga** contains the compressed texels on disk.

FIELD PRIMER

3D GRAPHICS CONCEPTS FOR THE SYSTEMS DEVELOPER

If your background is in backend services, networking, or databases rather than 3D graphics, game engine file formats can feel like a cryptic alphabet soup of vector math and arbitrary acronyms. This primer translates the essential 3D concepts into familiar systems-programming terms (structs, arrays, memory offsets, and directed graphs) before we dissect the binary archives.

0.1 GEOMETRY IN MEMORY: VERTICES, STRIDES & INDICES

In 3D engines, physical objects are not solid volumetric shapes; they are hollow boundary shells constructed from interconnected triangles (polygons).

- **Vertex (The C Struct):** A vertex is a single point on a 3D surface. In memory, it is a contiguous binary struct containing spatial coordinates, a normal vector (which direction the surface faces for lighting), and 2D texture coordinates:

```
struct Vertex32 {
    float x, y, z;           // 12 bytes: World position relative to mesh origin
    float nx, ny, nz;        // 12 bytes: Unit surface normal vector (used for lighting)
    float u, v;              // 8 bytes: Texture coordinates
}; // Total sizeof = 32 bytes
```

- **Stride (sizeof):** The byte width of each vertex struct. When parsing a raw binary buffer, the *stride* tells your unpacker how many bytes to jump to reach the next vertex. Refractor uses 32 bytes for standard meshes, and 40 bytes when a second UV set is present.
- **Index Buffer (Pointers into the Array):** Storing 3 full vertices for every triangle wastes immense memory because adjacent triangles share corners. Instead, the engine stores a flat array of unique vertices (the *Vertex Buffer*), and a separate array of integer offsets into that array (the *Index Buffer*). Every three consecutive indices define one triangle:

```
// Index buffer: two triangles forming a quad (indices 1 and 2 are shared)
uint16_t indices[] = { 0, 1, 2, 2, 3, 0 };
```

- **Primitives (List vs. Strip):** In a *Triangle List* (primitive 4), every triangle requires 3 separate indices. In a *Triangle Strip* (primitive 5), each new vertex after the first two creates a new triangle with the preceding two, saving memory and GPU bus bandwidth in 2002.

0.2 TEXTURING, UV COORDINATES & LIGHTMAPS

- **UV Coordinates:** A 2D texture image has horizontal (U) and vertical (V) axes normalized between 0.0 and 1.0. Assigning `u=0.5, v=0.5` to a 3D vertex pins that 3D corner to the exact center pixel of the texture bitmap.
- **Lightmaps (Static Offline Lighting):** In 2002, GPUs lacked the compute power to calculate dynamic soft shadows in real time. Level designers pre-calculated sunlight, ambient occlusion, and shadow penumbras offline using radiosity simulators. The resulting brightness values were saved into a secondary grayscale or colored texture called a **Lightmap**. At runtime, the GPU performs a dual-texture multiply:

```
PixelColor = DiffuseTexture(uv0) * LightmapTexture(uv1) * 2.0
```

FIELD PRIMER (CONTINUED)

RIGGING, GLTF & COORDINATE CONVENTIONS

0.3 THE SKELETAL RIGGING TRIO: .SKE, .SKN, AND .BAF

Rigid vehicles (tanks, jeeps) are simply parented trees of rigid parts. But an animated human soldier requires a three-tier deformation pipeline:

```
[1. .SKE (Skeleton Armature)] =====> [2. .SKN (Skin Weights)] <===== [3. .SM (3D Triangles)]
      |                               |
      v (Drives Joints)               v (Deforms Surface)
[4. .BAF (Animation Tracks)] =====> [FINAL POSED SOLDIER]
```

1. **The Skeleton / Armature (.ske)**: An abstract directed acyclic graph (DAG) of coordinate spaces called "bones" or "joints". It has no polygons, vertices, or textures. It is purely a parent-child hierarchy (`Pelvis` → `Spine` → `Clavicle` → `Forearm` → `Hand`) where each bone stores a 3x4 affine transform matrix representing its default rest position and rotation relative to its parent.
2. **The Skin Binding (.skn)**: The binding table that maps the flexible polygon mesh (the jacket, trousers) to the rigid skeleton bones. For every vertex, it stores an influence list:

```
Vertex #512: 75% Forearm Bone, 25% Elbow Bone
```

When the elbow bends, Vertex 512 moves as a weighted linear interpolation between the two bone transforms, allowing the elbow cloth to stretch naturally without tearing.

3. **The Animation Clip (.baf)**: A timeseries keyframe recording. It stores the rotational (quaternion) and positional offset for each bone at every frame. Refractor compresses these tracks using fixed-point math and Run-Length Encoding (RLE).

0.4 GLTF / GLB: THE "JSON + MEMCPY" WEB STANDARD

Why convert Refractor assets to **glTF 2.0 (GL Transmission Format)**?

- **Why Not Wavefront OBJ?** OBJ is a 1990s ASCII format. Parsing hundreds of thousands of floats from text strings in JavaScript on mobile CPUs burns battery and stalls the main browser thread.
- **Why Not Autodesk FBX?** FBX is a proprietary, closed binary container requiring huge C++ SDKs with complex internal object graphs.
- **The glTF Architecture:** glTF (and its single-file binary container `.glb`) separates metadata from raw payload. It contains a lightweight JSON header defining the scene graph, pointing directly into flat binary byte buffers (`BufferViews` and `Accessors`).
- **Direct VRAM Upload:** Crucially, the binary chunks inside a `.glb` match GPU vertex buffer layouts 1:1. The browser parses the JSON, reads the byte offsets, and hands the binary buffer directly to WebGL via `gl.bufferData()`. It is effectively a direct `memcpy` straight into GPU VRAM.

0.5 COORDINATE HANDEDNESS (THE 3D ENDIANNESS PROBLEM)

Just as CPUs differ in byte endianness (Little-Endian vs Big-Endian), 3D graphics engines differ in spatial **Handedness**:

- **Left-Handed (DirectX 8 / Refractor):** Point your left hand: Thumb right (+X), Index finger up (+Y), Middle finger *away from you* (+Z forward).
- **Right-Handed (OpenGL / WebGL / glTF / Three.js):** Point your right hand: Thumb right (+X), Index finger up (+Y), Middle finger *towards your face* (+Z backward).

If you import a Left-Handed model into WebGL without transforming its coordinates:

1. The model faces backwards along Z.
2. Its polygon winding order (the order vertices are traversed: clockwise vs counter-clockwise) is reversed. WebGL's back-face culling will hide the front of the model and render the interior!
3. Surface normal vectors point inside, causing lighting calculations to invert.

The Fix: Negate Z on all vertices and normals ($z \rightarrow -z$), and swap index pairs to reverse winding: $(i0, i1, i2) \rightarrow (i0, i2, i1)$.

CHAPTER 1

INSIDE THE REFRACTOR FLAT ARCHIVE (.RFA)

All game assets live in `.rfa` flat archives. The format is a container with an optional magic header, a contiguous data block, and an index table appended at the end of the payload.

OFFSET / FIELD	TYPE	DESCRIPTION & PARSING NOTES
<code>0x00</code> Header	<code>char[28]</code>	Optional magic string: <code>"Refractor2 FlatArchive 1.1 "</code> . If absent, base offset is 0.
<code>data_size</code>	<code>uint32_t</code>	Length of compressed/uncompressed payload chunk in bytes.
<code>compressed</code>	<code>uint32_t</code>	Payload compression flag (1 = compressed, 0 = raw).
<code>index_table</code>	(At EOF)	Located at <code>base_offset + data_size</code> . Contains entry count and directory records.

1.1 THE DIRECTORY RECORD & ENTRY UNPACKING

The index table at `base_offset + data_size` begins with `entry_count (uint32)`, followed by variable-length entry records. Each entry specifies its path, compressed/uncompressed sizes, and payload offset:

```
def read_entries(data: bytes, base_offset: int, data_size: int):
    pos = base_offset + data_size
    entry_count, = struct.unpack_from("<I", data, pos)
    pos += 4
    entries = {}
    for _ in range(entry_count):
        name_len, = struct.unpack_from("<I", data, pos)
        name = data[pos+4 : pos+4+name_len].decode("latin-1").replace("\\", "/")
        comp_size, uncomp_size, offset = struct.unpack_from("<III", data, pos+4+name_len)
        pos += 4 + name_len + 12 + 12 # 12 bytes metadata + 12 unused reserved bytes
        entries[name.lower()] = (offset, comp_size, uncomp_size)
    return entries
```

1.2 LZ01X SEGMENT DECOMPRESSION

Refractor compresses chunks using LZ01X. A compressed entry does not decompress in one single call; it begins with `segments_count (uint32)`, followed by per-segment descriptors:

```
import ctypes
_lzo = ctypes.CDLL("liblzo2.so.2")

def decompress_entry(raw_entry_data: bytes, uncompressed_size: int) -> bytes:
    segments_count, = struct.unpack_from("<I", raw_entry_data, 0)
    pos = 4
    out_buf = ctypes.create_string_buffer(uncompressed_size)
    out_offset = 0
    for _ in range(segments_count):
        seg_csize, seg_usize, seg_offset = struct.unpack_from("<III", raw_entry_data, pos)
        pos += 12
        chunk = raw_entry_data[seg_offset : seg_offset + seg_csize]
        written = ctypes.c_ulong(seg_usize)
        _lzo.lzo1x_decompress(chunk, seg_csize,
                             ctypes.byref(out_buf, out_offset), ctypes.byref(written), None)
        out_offset += seg_usize
    return bytes(out_buf)
```

[!] GOTCHA: THE LINUX CASE-SENSITIVITY LANDMINE

Battlefield 1942 was authored on Windows where filesystems are case-insensitive. Mod authors freely mixed casing: `GCMOD/Archives` vs `EoD/archives` (all lowercase) vs `interstate/archives`. A naive Linux `mod_dir / "Archives"` lookup returns empty, silently skipping entire mods. Always perform a case-folded directory scan:

```
def find_archives(p): return next((c for c in p.iterdir() if c.name.lower() == "archives"), None)
```

1.3 MOD INHERITANCE & MONOLITHIC PACKS

Refractor mods inherit content hierarchically. An archive never duplicates vanilla assets; its `init.con` declares search paths using `game.addModPath`:

```
FHSW (child mod) ---> FH (parent mod) ---> Bf1942 (base game)
```

Resolution Rule: Always probe nearest-child first, falling back to base vanilla. When extracting levels, **underlay** parent archives: Desert Combat's `Gazala.rfa` only defines vehicle spawns; it relies on vanilla `bf1942/levels/Gazala.rfa` for control points, flags, and terrain size.

Monolithic Packs: Total conversion mods like FHSW frequently concatenate hundreds of vehicle scripts into a single monolithic `!_PACK_ALLIES/Compressed.con` file. The parser unpacks these virtual trees by detecting comments injected by mod packers: `rem folder = <Name>` and `rem sauce = <File>`.

1.4 THE MISSING TEXTURE.RFA CASE STUDY

During early pipeline testing, vanilla vehicles extracted as bare untextured geometry. Texture resolution stalled at a dismal **6.2%**. The code was inspected repeatedly for path bugs, but the fault lay in data: the Wine test install was missing base `texture.rfa` (98,885,015 bytes, 1,605 entries).

Once restored to the archive search pool, resolution surged to **96.0%**. The pipeline maintains an `ArchivePool` that searches across open archives in exact engine priority:

```
# Priority order for texture resolution
1. Objects.rfa (Mod override: Objects/<path>/Art/<mesh>.rs)
2. StandardMesh.rfa (Global mesh shader: StandardMesh/<mesh>.rs)
3. Texture.rfa (Base vehicle & weapon textures: texture/<name>.dds)
4. Menu.rfa (HUD icons and kit selection graphics)
```

1.5 ARCHIVE POOLING & FILE DESCRIPTORS

Extracting 1,000 models across 14 mods touches tens of thousands of archive lookups. Naive per-file archive opens quickly trigger `EMFILE (Too many open files)`. The `ArchivePool` class opens each `.rfa` exactly once, parses and caches the index table in memory with normalized lowercase paths, and reuses memory-mapped views for zero-copy slice decompression.

CHAPTER 2

PARSING A PARTICULAR ASSET: STEP-BY-STEP

Let us trace the complete extraction of a single asset: the **Sherman tank**. We begin with `Objects.con` and step through the binary formats until we reach textured 3D polygons.

STEP 1: SCRIPT REPLAY & SCENE GRAPH (OBJECTS.CON)

Refractor scripts are imperative command streams. As the engine executes each line, it maintains an active template context:

```
ObjectTemplate.create Bundle ShermanComplex
ObjectTemplate.geometry Sherman_Hull_M1
ObjectTemplate.addTemplate ShermanTower
ObjectTemplate.setPosition 0/-0.8/0    <-- WARNING: THE TRAP IS HERE
```

[!] THE `SETPOSITION` CHILD INSTANCE TRAP

This single line cost us a full day of debugging.

In Refractor syntax, `setPosition` and `setRotation` immediately *after* an `addTemplate` apply to the **child instance just added**, NOT to the parent bundle! Before any `addTemplate`, they configure the parent. If your parser assigns `setPosition` to the current enclosing template, every child part collapses to `(0, 0, 0)`. On screen, the turret vanishes inside the hull.

Accumulators vs. Holds (The Engine Lean Bug)

Vehicle rotational parts declare player inputs via `setInputToYaw`, `setInputToPitch`, and `setInputToRoll`. They pass either symbolic constants (`c_PIThrottle`) or raw integers (`4` = MouseX, `5` = MouseY). Whether an engine axis is a continuous spinning rate or a fixed pose is decided by its **declared span**:

- `span 2250..15000`: Aircraft propellers, helicopter rotors, and wheels — continuous accumulators.
- `span 2.0`: Tank hulls — a $\pm 1^\circ$ chassis lean under acceleration and steering.

[!] WAR STORY: THE ORBITING TANK RUNNING GEAR

Early readers treated every template of kind `Engine` as a continuous spin rate. On a Sherman or Tiger, the hull lean was integrated as a continuous angular rate. Because the tracks and road wheels are children of the Engine, **the entire suspension and running gear slowly orbited the hull in mid-air**. We introduced `ACCUMULATOR_SPAN = 360.0`: anything below 360° is a clamped pose, not a spin.

STEP 2: GEOMETRY TEMPLATE INDIRECTION

`ObjectTemplate.geometry Sherman_Hull_M1` does not load `Sherman_Hull_M1.sm` directly. It queries `Geometries.con`:

```
GeometryTemplate.create StandardMesh Sherman_MGun_Mount_M1
GeometryTemplate.file Tank_MGun_Mount_M1
```

Here, the template name differs from the file on disk. The parser must maintain a dictionary of `GeometryTemplates` before touching the filesystem.

STEP 3: DISSECTING THE BINARY MESH (.SM)

The `.sm` format stores rigid indexed triangle meshes and collision geometries.

BYTE OFFSET	TYPE	NAME	DESCRIPTION
0x00	uint32	version	Universal version identifier: 9 or 10.
0x04	uint32	unknown	Always 0.
0x08 - 0x1F	float[6]	boundsMin/Max	Bounding box (xmin, ymin, zmin) to (xmax, ymax, zmax).
0x20	uint8	qflag	Present only if <code>version > 9</code> .
Var	uint32	colCount	Number of collision hulls embedded in the file.
Var	uint32	lodCount	LOD chain depth (1 for weapon parts, 6 for vehicles).

The Material Descriptors vs. Payload Trap

In each LOD, materials are serialized with a counter-intuitive memory layout:

```
[LOD Header] -> [Mat 0 Desc] -> [Mat 1 Desc] -> [Mat 0 Verts] -> [Mat 0 Indices] -> [Mat 1 Verts] -> [Mat 1 Indices]
```

All material descriptors come first; all binary vertex and index payloads follow afterwards in matching sequence.

```
# Material Descriptor Layout
name_len, = struct.unpack_from("<I", data, pos)
name = data[pos+4 : pos+4+name_len].decode("latin-1")
pos += 4 + name_len
u0, u1, u2, prim, flags, stride, vcount, icount, u3 = struct.unpack_from("<9I", data, pos)
pos += 36
```

Vertex Strides

Refractor uses two universal vertex layouts:

- **Stride 32 bytes** (Standard): Position ($3 \times \text{f32}$) + Normal ($3 \times \text{f32}$) + UV ($2 \times \text{f32}$).
- **Stride 40 bytes** (Static Lightmapped): Position (3f) + Normal (3f) + Diffuse UV0 (2f) + Lightmap UV1 (2f).

DirectX 8 Index Buffers & Primitives

- `primitive == 4`: Triangle List. Indices grouped as (i0, i1, i2).
- `primitive == 5`: Triangle Strip. Unwound with alternating winding order:

```
for i in range(len(idx) - 2):
    a, b, c = idx[i], idx[i+1], idx[i+2]
    if a != b and b != c and a != c:
        tris.append((a, c, b) if i % 2 else (a, b, c))
```

[!] GOTCHA: THE SIGNED INT16 OVERFLOW

DirectX 8 index buffers use unsigned 16-bit integers (`uint16_t` / struct format `<H`). If you unpack indices as signed shorts (`<h`), any large warship (Yamato, Prince of Wales) or complex industrial bunker with over 32,767 vertices overflows into negative numbers. The geometry explodes into a spiderweb of stretched polygons across the world origin.

STEP 4: SHADERS & TEXTURES (.RS, .DDS, .TGA)

The material name in the `.sm` (e.g. `Sherman_Hull_M1_Material0`) is looked up in `StandardMesh/Sherman_Hull_M1.rs` or an override `Objects/.../Art/Sherman_Hull_M1.rs`:

```
subshader "Sherman_Hull_M1_Material0" "StandardMesh/Default" {
    texture "texture/sherma_I";
    alphaTest 0.5;
    twosided true;
}
```

Notice that texture paths carry no file extension. The Refractor engine probes `.dds` first, then falls back to `.tga`.

DDS Block Formats & Alpha Dilation

Textures are compressed using DXT block compression:

- **DXT1**: 4x4 pixel blocks, 1-bit alpha or 4-color opaque punchthrough.
- **DXT3**: Explicit 4-bit alpha channel per texel. Used for complex cutouts.
- **DXT5**: Interpolated 8-step alpha ramp. Used for smooth gradients and smoke.

Alpha Bleeding: Cutout textures (foliage, track links, wire fences) often store black (0, 0, 0) in transparent texels. When Three.js bilinearly filters edges between opaque and transparent pixels, the black color bleeds into the boundary, creating an ugly dark fringe. We run an alpha dilation pass on export: copy RGB from neighboring opaque pixels into any texel with $\alpha \leq 16$.

STEP 5: GLTF SCENE ASSEMBLY & VERIFICATION

In `bf42/assemble.py`, the parsed template tree is recursively compiled into a glTF 2.0 binary container (`.glb`). A few critical rules govern this stage:

- **Multi-Material Parts**: A single `.sm` part with multiple materials is mapped to a single glTF `Node` with one `Mesh` containing multiple `Primitives`, preserving local part hierarchy.
- **Collision Mesh Isolation**: Collision layers embedded in the `.sm` are stripped from the render scene and stored under `mesh.extras.collision` to prevent collision hulls from drawing over the vehicle body.
- **Automated Health Triage**: The pipeline audits every extracted model, stamping a report into `models.json`:
 - `CLEAN`: 100% textures resolved, valid bounding box, intact node tree.
 - `DEGRADED`: Non-critical missing textures (e.g. secondary lightmap).
 - `BROKEN`: Missing primary geometry or corrupt index buffers.

CHAPTER 3

SKELETONS, SKINS & ANIMATION POSES

Vehicles are assembled by chaining `setPosition` commands. Hand weapons and soldiers are entirely different: a weapon script contains **no position coordinates at all**. It declares:

```
ObjectTemplate.bindToSkeletonPart Bip01_R_Hand
```

The skeleton is the placement. The file that animates the reload bolt is the exact file that positions the bolt on the rifle.

3.1 THE SKELETON FORMAT (.SKE)

A `.ske` file defines the rest-pose bone hierarchy.

```
uint32 version (1)
uint32 boneCount
per bone:
  uint16 nameLen (including trailing NUL)
  char   name[nameLen]
  int16  parentIndex (-1 for root)
  float  matrix[12] (Row-major 3x4: [R00 R01 R02 Tx, R10 R11 R12 Ty, R20 R21 R22 Tz])
```

[★] THE Z-MIRROR CONJUGATION TRAP

A `.ske` is stored mirrored in Z against the `.sm` mesh it poses!

In `UsSoldier.ske`, `Bip01 Head` sits at $z = -1.51$ m, while the head mesh it drives sits at $z = +1.56..1.87$ m. If you read the skeleton raw:

- The K98 bolt spawns behind the wooden butt plate.
- The Colt 1911 magazine floats mid-air behind the grip.
- 15% to 60% of every weapon sub-part lands completely outside the silhouette of the gun!

The Mathematical Fix: A mirror is a change of basis $S = \text{diag}(1, 1, -1)$. It conjugates both rotation and translation:

```
def conjugate_ske(rot, trans):
    rot_c = ((rot[0][0], rot[0][1], -rot[0][2]),
              (rot[1][0], rot[1][1], -rot[1][2]),
              (-rot[2][0], -rot[2][1], rot[2][2]))
    trans_c = (trans[0], trans[1], -trans[2])
    return rot_c, trans_c
```

Applying this conjugation at parse time brings silhouette error from 60% down to under 3%.

3.2 VERTEX SKINS (.SKN) & RIGID MAP RECOVERY

Soldier models consist of three separate meshes: body, head, and hands. Body and head share an origin; hands do not — they were exported in an arbitrary Max workspace coordinate frame.

A `.skn` file records vertex bind influences:

```
float restPos[3]
uint8 influenceCount
per influence:
  uint16 boneIndex, float weight, float boneLocalOffset[3]
```

At bind pose: `rest = R_bone * offset + T_bone`.

How to weld detached hands: The hand mesh is not in the soldier body skin, but both share the `forearm` bone. We find vertices with `weight == 1.0` exclusively on the forearm bone, compute the rigid transformation between the body's forearm space and the hand's forearm space, and snap the floating hands directly onto the sleeve.

3.3 BONE ANIMATION CLIPS (.BAF)

Battlefield 1942 poses soldiers by blending two clips: a weapon-independent lower body clip (`Lb_Stand`) and a weapon-specific upper body clip (`3PStandAimUpperThompson.baf`).

HEADER FIELD	TYPE	RULE / SPECIFICATION
version	uint32	Always 3 in vanilla.
boneCount	uint16	Number of animated bones.
frameCount	uint32	Total animation frame count.
precision	uint8	Fraction bits for position channels ONLY.

The 7 Channels & RLE Compression

Each bone stores 7 channels: Quaternion (x, y, z, w), followed by Position (x, y, z). Channels are stored as 16-bit signed fixed-point integers compressed via Run-Length Encoding:

- `control & 0x80`: Hold run. Read 1 value word, repeat for `control & 0x7F` frames.
- Else: Literal run. Read `control & 0x7F` distinct words.

[!] GOTCHA: THE PRECISION BIT SHIFT TRAP

The header byte `precision` tells you how many fractional bits exist in position values: `pos = val / 2^precision`. **It does NOT apply to quaternions!**

Quaternions are *strictly* fixed-point 1.15: `quat = val / 32768.0`.

In 1,121 vanilla clips, `precision == 15`, which tricked early community tools into assuming the precision byte applied to all channels. The 33 clips that drop to precision 11–14 (parachuting, ladder climbing, death falls) broke completely: dividing quaternions by 2^{11} made them 16x larger than unit norm!

The Shattered Hand & The 180° Yaw

A `.baf` clip's transforms replace the skeleton's local transforms outright. However, two conventions must be accounted for:

1. **Quaternion Transposition:** Refractor stores row-vector quaternions. To convert to standard column-vector matrices, you must take the **quaternion conjugate**: $(x, y, z, w) \rightarrow (-x, -y, -z, w)$.
Failing to conjugate produces a shattered hand: near-identity leg rotations absorb the error, but fingers and wrists rotate backwards into a mangled ball of flesh.
2. **Root Track 180° Yaw:** The clip coordinate space is rotated 180° around Y relative to mesh world space. The skeleton root joint must be pre-multiplied by `Ry180 = diag(-1, 1, -1)`.

3.4 THE STATE MACHINE REPLAY & WEAPON GRAFTING

Nothing on a weapon template states which animation clip to play. The resolution requires replaying `animations/AnimationStates.con`:

- `copyState2` commands dynamically substitute the weapon name into state strings.
- The six-argument `copyState` specifies donor clips: the K98 has no 3P aim clip of its own and borrows the British Lee-Enfield No. 4; the Panzerschreck borrows the Bazooka; the Walther P38 borrows the Colt 1911.
- **Grafting the Weapon:** Every hand weapon skeleton is rooted at a bone literally named `Bip01_R_Hand`. Attaching the weapon is a simple joint grafting identity:

```
weapon_world = posed(Bip01_R_Hand) * rest(root)^-1 * rest(main)
```

CHAPTER 4

RECONSTRUCTING THE WORLD IN THE BROWSER

Once meshes, textures, skins, and animations are verified, we export them to binary glTF (`.glb`) and reconstruct the battlefield inside WebGL using Three.js.

4.1 THE HANDEDNESS BRIDGE

Refractor is a **Left-Handed** coordinate system (+X right, +Y up, +Z *forward*). WebGL and glTF are **Right-Handed** (+X right, +Y up, -Z forward).

Rather than slapping a `scale(1, 1, -1)` on the root node (which inverts polygon winding, corrupts back-face culling, and flips normal vectors), we bake the coordinate conversion into the geometry pipeline on export:

```
# In gltf.py: baking handedness conversion
positions = [(x, y, -z) for (x, y, z) in sm.positions]
normals    = [(nx, ny, -nz) for (nx, ny, nz) in sm.normals]
indices    = [idx for i in range(0, len(raw_indices), 3)
              for idx in (raw_indices[i], raw_indices[i+2], raw_indices[i+1])] # Reverse winding
```

For vehicle Euler rotations (Yaw, Pitch, Roll): `R = Ry(-yaw) * Rx(-pitch) * Rz(roll)`. Mirroring Z reverses the direction of rotations around X and Y, but leaves rotation around Z invariant.

4.2 TERRAIN RECONSTRUCTION: HEIGHTMAPS & TILES

BF1942 terrains are defined by:

- `Heightmap.raw`: 16-bit unsigned elevation grid sampled on a 4-meter spacing:

$$\text{Elevation (Y)} = (\text{sample} / 65535) * 256 * \text{yScale}$$
- `Tx.dds`: 256-meter square surface color tiles.

[*] THE PRE-LIT TERRAIN TRAP

Battlecraft (the 2002 level editor) **pre-baked sunlight, ambient color, and static shadows directly into the Tx tile image pixels**. If you look closely at any desert tile, building shadows and treeline cast shadows are already painted into the texture.

The original engine rendered terrain with `MeshBasicMaterial` (flat unlit) and fog. In our first WebGL prototype, we attached standard Three.js directional sun and hemisphere lights to the scene. **The terrain was lit twice**. On Tobruk, it looked acceptable because desert sand is warm. But on Wake Island, the cool blue sky hemisphere multiplied the baked light, turning the sunny Pacific sands into a dead, dark grey smear.
Fix: Terrain meshes must use unlit basic materials!

4.3 THE MODULATE-2X DETAIL SHADER

Refractor terrains achieve fine grain underfoot by tiling a 512px detail texture 16 times across each 256m tile:

```
Final Color = clamp(2.0 * TileColor * DetailColor, 0.0, 1.0)
```

Mid-grey (`128, 128, 128`) is the neutral identity: darker texels carve dirt ruts; brighter texels highlight sand crests.

[!] THE SRGB GAMMA CURVE DISASTER

In DirectX 8, `modulate-2x` was computed in raw 8-bit display space. Modern WebGL runs a linear color pipeline. When Three.js decoded the detail map into linear space, neutral grey 0.5 became $0.5^2 \approx 0.25$. Multiplying by 2 gave 0.5 instead of 1.0 — **the entire terrain became 2x too dark**.

The Solution: Set `tex.colorSpace = THREE.NoColorSpace` and perform the `modulate-2x` combine in sRGB display space via an `onBeforeCompile` GLSL hook:

```
// Three.js onBeforeCompile fragment patch
diffuseColor.rgb = bfFromSrgb( min( vec3(1.0),
2.0 * bfToSrgb( diffuseColor.rgb ) * texture2D( tDetail, vMapUv * detailRepeats ).rgb ) );
```

4.4 STATIC MESH LIGHTMAPS & THE FLIP-Y BUG

Buildings, bunkers, and bridges use Stride-40 vertices with a second UV set (`uv1`) mapped to a level lightmap atlas (e.g. `radarbun_m1_406-75-761.png`).

```
+-----+
| THE LIGHTMAP TRIAGE LOG |
| 1. MISSING PASS:  UserData was attached to the parent glTF Group, but |
|                   | the lightmap loop only checked direct Mesh instances. |
| 2. BURNT SHELLS:  Lightmap was tagged sRGB. A lightmap is a scalar |
|                   | multiplier, not a color. Linear decoding squared darks! |
| 3. UPSIDE DOWN:   glTF images load with flipY=false. THREE.TextureLoader |
|                   | defaults to flipY=true. The lightmap was sampled |
|                   | mirrored in V, projecting rooftop shadows onto floors! |
+-----+
```

Injecting the Ambient Floor: The lightmap bake contains direct sun light only. In the Refractor engine, static objects receive the level's ambient light floor:

```
// Inside patched MeshBasicMaterial fragment shader
vec3 lmTexel = texture2D( lightMap, vLightMapUv ).rgb;
reflectedLight.indirectDiffuse += diffuseColor.rgb * max( lmTexel * 2.0, ambientFloor );
```

4.5 SOLDIER STANCE CROSSFADING

Rather than exporting three separate glTF files for standing, crouching, and prone postures, each extracted `<Soldier>__<Weapon>.pose.glb` packages all three stances as 1-frame animation clips:

- `stand`: Driven by `Ub_StandAim<Weapon>` + `Lb_Stand`
- `crouch`: Driven by `Ub_Crouch<Weapon>` + `Lb_Crouch`
- `lie`: Driven by `Ub_Lie<Weapon>` + `Lb_Lie`

In the browser, clicking between stances simply triggers a Three.js `AnimationMixer` crossfade over 0.25 seconds:

```
function setStance(newAction) {
    currentAction.fadeOut(0.25);
    newAction.reset().fadeIn(0.25).play();
    currentAction = newAction;
}
```

Because every clip animates the exact same union of 42 bones, bone matrices interpolate smoothly without popping, tearing, or dropping the weapon from the soldier's hands.

CHAPTER 5

BALLISTICS, FIRING EFFECTS & SPATIAL AUDIO

A static vehicle in a viewer is only half the battle. Bringing the battlefield to life requires two dynamic systems: simulating weapon fire (muzzle flashes, tracers, ballistic projectiles, and barrel recoil) and extracting authentic spatial soundscapes.

5.1 HOW THE ENGINE DEFINES A SHOT (TYPED PROJECTILES)

Every weapon is declared via an `ObjectTemplate.create Firearms <Name>` script. In shipped data, firearms specify spatial muzzle locations via `addFireArmsPosition` (including wing-gun convergence yaw/pitch/roll), rate of fire (`roundOffFire`), and a linked `projectileTemplate`.

Projectiles come in three distinct, data-driven archetypes:

- **Bullets (Invisible Kinetic Rounds):** Bullets have no 3D geometry in flight. Visibility comes entirely from `setTracerTemplate <tpl> CRD_NONE/<interval>/0/0`. Every N-th round spawns a `TLight_m1` tracer cylinder stretched by `tracerScaler` (e.g. a Spitfire fires 12 rounds/sec and draws a tracer every 3rd round). Between tracer rounds, the game draws *nothing*.
- **Shells (Ballistic Geometry):** Heavy rounds have physical 3D geometry and fly it: a Sherman fires `projectile_m1` ($0.115 \times 0.115 \times 0.48$ m) at 100 m/s, governed by gravity drop and pulling a smoke trail (`e_PanzShootTrail`).
- **Rockets (Self-Propelled Accelerators):** A rocket is a shell containing a child Engine with `setEngineType c_ETRocket`. For example, the Katyusha's 2.04m rocket (`r_rocket_m1`) launches at a muzzle velocity of only 45 m/s, accelerating over its flight path while emitting a looping smoke plume (`e_KatyushaFume`).

[!] THE KATYUSHA FAINT STREAK BUG

Early versions of `assemble.py` exported the projectile simply as a template name string. The browser viewer ignored the string and fell back to rendering a generic tracer streak for every weapon with `velocity > 0`. As a result, **the Katyusha's massive 2-meter rockets rendered as faint 2-meter bullet streaks!**

The Fix: The pipeline now inspects the template tree, detects child `c_ETRocket` engines, bakes the projectile mesh (preferring `visibleDummyProjectileTemplate`) as a hidden node in the glTF, and simulates acceleration ramps and gravity in the WebGL tick loop.

5.2 MUZZLE FLASHES, EMITTER DRIFT & RECOIL

Muzzle flashes in Refractor are `EffectBundle` trees composed of particle meshes and 2D billboard sprites rendered with additive blending (`destBlendMode BMOne`).

- **The Emitter Motion Trap:** Authors did not just spawn a static flash at the muzzle tip. Scripts declare `positionalSpeedInDof` and `relativePositionInDof`, drifting smoke and flares *backward along the barrel* (e.g. Sherman smoke drifts at -5 m/s while the flash peaks at -1 m/s). If you park emitters statically at the muzzle, muzzle smoke reads as an absurdly gigantic 3.6-meter flame ball.
- **Scale Clamping:** Authored size ramps (`sizeOverTime`) often ramp to extreme multipliers (up to 9.4× on heavy guns). In WebGL, we clamp node flash ramps to `FLASH_RAMP_MAX = 3.0` to prevent flashes from dwarfing the vehicle.
- **Barrel Recoil:** Guns declare `recoilSpeed` and `recoilLength`. On firing, the barrel node kicks backward along its local axis and smoothly recovers:

```
barrel.position.z = -recoilLength * (1.0 - recoilProgress);
```

CHAPTER 5 (CONTINUED)

EXTRACTING SOUNDS & SPATIAL AUDIO

5.3 THE REFRACTOR AUDIO ARCHITECTURE (.SSC)

Battlefield 1942 stores audio in `Sound.rfa` and level archives as uncompressed 16-bit PCM WAV files (typically 22.05 kHz or 44.1 kHz, mono/stereo). Sound playback rules are governed by **Sound Script Configuration** (`.ssc`) files:

```
newPatch
load @ROOT/Sound/@RT/Water_waves.wav
loop
minDistance 10.0
maxDistance 150.0
volume 0.8
priority 5
```

Ambient & Shoreline Area Sounds

Audio in levels is structured into two main layers:

1. **Global Ambient Loops:** Declared in `Sounds/Environment.con`, linking to `Environment.ssc` (e.g. `wind`, `windall`, `windcalm`). This is a non-positional background loop.
2. **Positional Shoreline & River Audio:** Declared in level `Sounds.con` as `AreaObject` or `SimpleObject` polylines. Instead of a single point source, sounds like surf and river rapids are defined as 2D/3D polylines tracing the shoreline. The sound emanates from the *closest point on the polyline to the player's camera*.

5.4 WEB AUDIO API SPATIALIZATION & BUG TRIAGE

In the browser viewer (`viewer/map.html`), audio is driven by the modern **Web Audio API** using `AudioContext` and `PannerNode` (HRTF spatialization).

[★] WAR STORY: THE WAKE ISLAND "HALL ECHO"

When shoreline audio was first enabled, flying over Wake Island produced a disorienting, hollow "slapback echo", as if standing in an empty cathedral.

The Investigation:

- Wake Island defines 3 separate shoreline polylines that all play the exact same file: `water_waves.wav` (a 2.54-second mono loop). Two polylines were within earshot at spawn.
- Each polyline spawned its own independent `PositionalAudio` node.
- Crucially, the file loader included a development cache-buster query parameter (`?t=Date.now()`). This defeated the browser's audio decode cache, causing the two streams to decode and start playing **50 to 200 ms apart**.
- Two identical audio loops playing 100 ms out of phase through 3D HRTF panners created a severe acoustic comb filter / slapback echo!

The Fix: Deduplicate voices! The engine now maintains **one looping voice per unique WAV file** across all polylines in a group. At every animation frame, it calculates the camera's closest distance across the union of all polylines and updates the single voice's position and gain.

[!] THE DOUBLE-ATTENUATION TRAP

Web Audio's `PannerNode` applies an inverse distance attenuation model by default. When combined with the game's authored `minDistance` / `maxDistance` linear volume ramps from the `.ssc` file, **the audio was attenuated twice**, cutting out shoreline surf just a few meters from the water.

The Solution: Set `panner.setRolloffFactor(0)`. This disables Web Audio's automatic distance dropoff, allowing the game's authentic `.ssc` volume ramp to control the gain curve exclusively.

5.5 DISCOVERY QUIRKS: SOUND/ VS SOUNDS/

While vanilla levels store environment audio under `Sounds/` (plural), historical level archives contain quirks:

- `Kasserine_Pass` stores its sound config under `Sound/` (singular) and ships a custom `Sound/ambfx.wav` inside the level archive. The extractor implements a case-insensitive fallback to `Sound/`.
- `Truk` contains no sound scripts at all — the only genuinely silent map in vanilla Battlefield 1942.

APPENDIX

FIELD QUICK-REFERENCE CHEAT SHEET

MAGIC STRINGS & EXTENSIONS

EXTENSION	DOMAIN	MAGIC / FORMAT	GOTCHA / REFERENCE NOTE
<code>.rfa</code>	Archive	<code>Refractor2 FlatArchive 1.1</code>	Trailing index table; LZ01X segment decompression.
<code>.sm</code>	StandardMesh	<code>version == 9 10</code>	Descriptors first, payloads second. Stride 32/40. Unsigned 16-bit indices.
<code>.rs</code>	Shader	ASCII script	<code>subshader</code> vs <code>shader</code> . Maps material names to texture stages.
<code>.ske</code>	Skeleton	<code>version == 1</code>	3x4 parent-relative matrices. Stored mirrored in Z (R → SRS).
<code>.skn</code>	Skin Weights	<code>version == 1</code>	Rest pos + 1–2 bone-local offsets. Exclusive pairs recover rigid transforms.
<code>.baf</code>	Bone Anim	<code>version == 3</code>	7-channel RLE. Precision byte is for position only! Quaternions conjugated.
<code>.con</code>	Scripts	ASCII script	<code>setPosition</code> applies to preceding <code>addTemplate</code> child!
<code>.ssc</code>	Audio Config	ASCII script	Near/far distance ramps, loop flags, and WAV sample links.

COORDINATE TRANSFORMATION MATRIX

COORDINATE SPACE	HANDEDNESS	AXES CONVENTION	CONVERSION TO WEBGL GLTF
Refractor Engine	Left-Handed	+X Right, +Y Up, +Z Forward	$X \rightarrow X, Y \rightarrow Y, Z \rightarrow -Z$
gITF 2.0 / WebGL	Right-Handed	+X Right, +Y Up, -Z Forward	Native target. Triangle index winding reversed.
Refractor .ske	Mirrored Left	+X Right, +Y Up, -Z Forward	Conjugate: $R \rightarrow SRS, t \rightarrow St$ where $S = \text{diag}(1, 1, -1)$.
Refractor .baf	Yawed Left	+X Left, +Y Up, -Z Back	Conjugate quat: $(-x, -y, -z, w)$; Root joint yawed 180° (Ry180).

TRIAGE CHECKLIST FOR EXTRACTION BUGS

SYMPTOM ON SCREEN	ROOT CAUSE	THE ONE-LINE FIX
Turret / gun missing or at (0,0,0)	<code>setPosition</code> assigned to parent template instead of child	Bind transform to preceding <code>addTemplate</code> instance
Tank wheels orbiting the hull	Chassis 1° roll lean treated as continuous engine rate	Enforce <code>ACCUMULATOR_SPAN = 360.0</code> threshold
Warship / bunker polygon spiderweb	DirectX 8 16-bit indices unpacked as signed short	Change struct unpack format from <code><h</code> to <code><H</code>
Rifle bolt floating behind butt plate	<code>.ske</code> rest matrix read without Z conjugation	Conjugate: $R' = S^*R^*S, t' = S^*t$ with $S = \text{diag}(1, 1, -1)$
Soldier hand twisted into shredded meat	<code>.baf</code> quaternion read without conjugation	Conjugate quaternion: $(-x, -y, -z, w)$
Terrain renders 2.3x too dark	Modulate-2x detail map decoded to linear color space	Set <code>NoColorSpace</code> and combine in sRGB display space
Building rooftop shadows on floor	TextureLoader inverted V on lightmap atlas	Set <code>lightmap.flipY = false</code>
Katyusha fires faint streak instead of rocket	Projectile exported as bare name string; typed as bullet	Bake dummy projectile mesh; simulate acceleration ramp
Muzzle smoke appears as gigantic flame	Additive smoke emitter parked statically at barrel tip	Apply <code>positionalSpeedInDof</code> drift; clamp flash ramp to 3×
Shoreline waves sound like hall echo	Duplicate voices playing across polylines with desynced cache	Deduplicate to 1 looping voice per unique WAV; zero panner rolloff